



경구약제 이미지 객체 검출

프로젝트

최종 결과 보고서

프로젝트 팀 구성

Project Manager

오병주

프로젝트 총괄, 전략 수립 및 방향 제시, 발표 및 문서화 담당

Data Engineer

이상윤

데이터 EDA 및 전처리, 데이터 문제 발견, Segmentation 관련
진행 담당

Model Architect

서준범

BBox 파이프라인 및 Baseline 모델 구축, 표준화된 실험 환경
설계, 최종 아키텍처 결정

Experimentation Lead

김승우

하이퍼파라미터 튜닝, 데이터 증강 실험, 추가 훈련셋 제작

프로젝트 여정: 6단계

구상



EDA 및 데이터 오류 발견

데이터 탐색 및 문제 정의



BBox 파이프라인 및 실험

Baseline 모델 구축



모델 성능 평가

모델 성능 증진 실험



BBox 한계 발견

데이터 누수 확인



Segmentation 전략 전환

핵심 솔루션 개발



시스템화

API 서버 및 실험

EDA 실험결과 및 분석

EDA (클래스 불균형 - TOP 20 클래스 카운트 및 점유율)

전체 73개의 클래스중 위 클래스 20개가 전체 라벨에서 얼마나 비중이 높은지를 보여줍니다

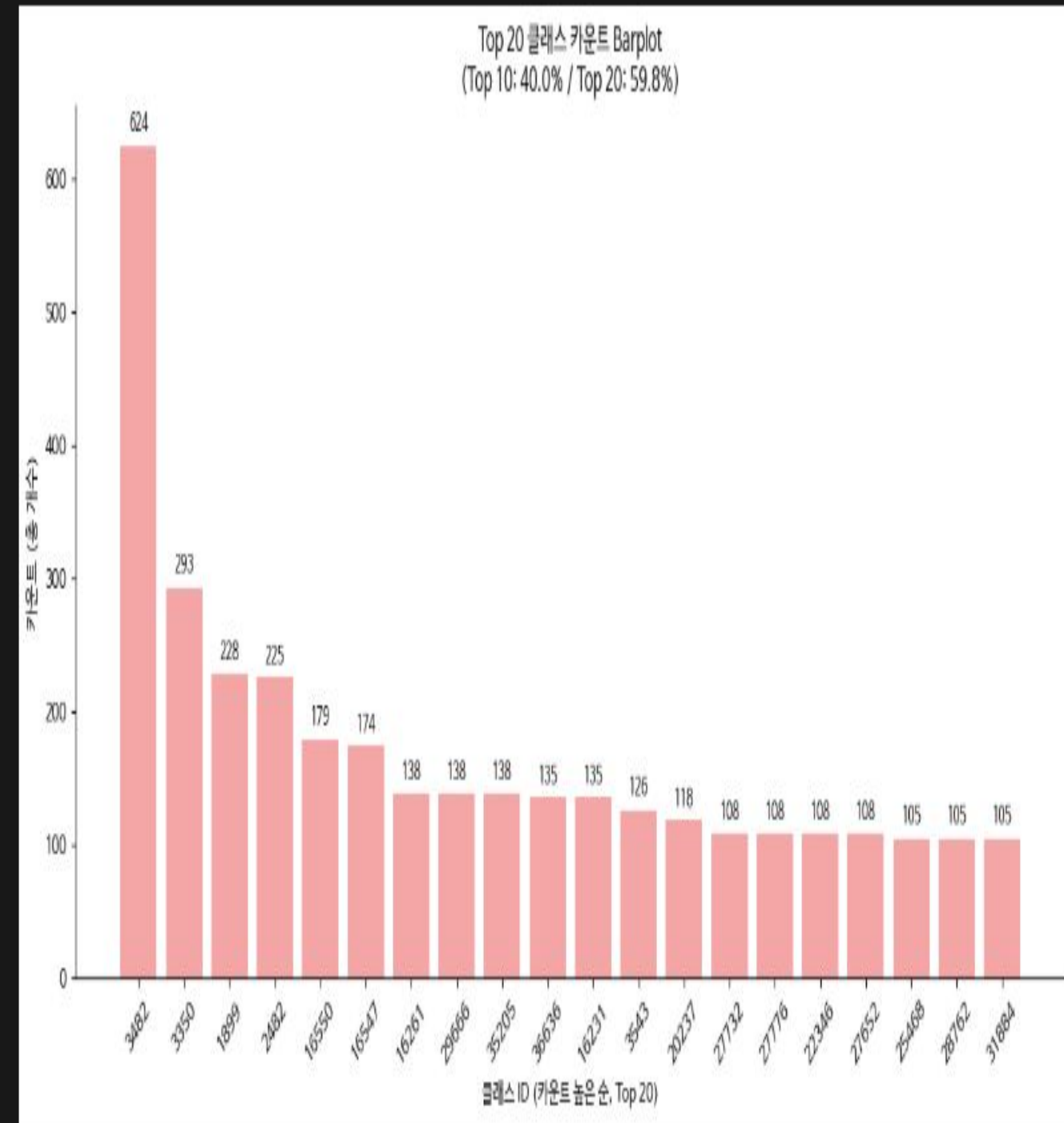
심각한 데이터 불균형 확인: 총 73개 클래스 중 소수 클래스에 데이터가 집중됨.

주요 분석 결과 :

- 상위 10개 클래스가 전체 데이터의 **40%** 차지
- 상위 20개 클래스가 전체 데이터의 약 **60%** 차지

시사점 :

- 이는 훈련 모델의 편향(bias)을 유발할 수 있음.
- 따라서 **Class Loss** 가중치를 **상향 조정** 하여 훈련 전략을 수정할 필요가 있음.



Top 10 클래스 점유율: 40.0% (총 2272개)

Top 20 클래스 점유율: 59.8% (총 3398개)

EDA 실험결과 및 분석

EDA (클래스 불균형 - 상위/하위 TOP 20 클래스 카운트)

상위 20개의 클래스와 하위 20개의 클래스의 불균형을 알수 있도록 시각화 하였습니다

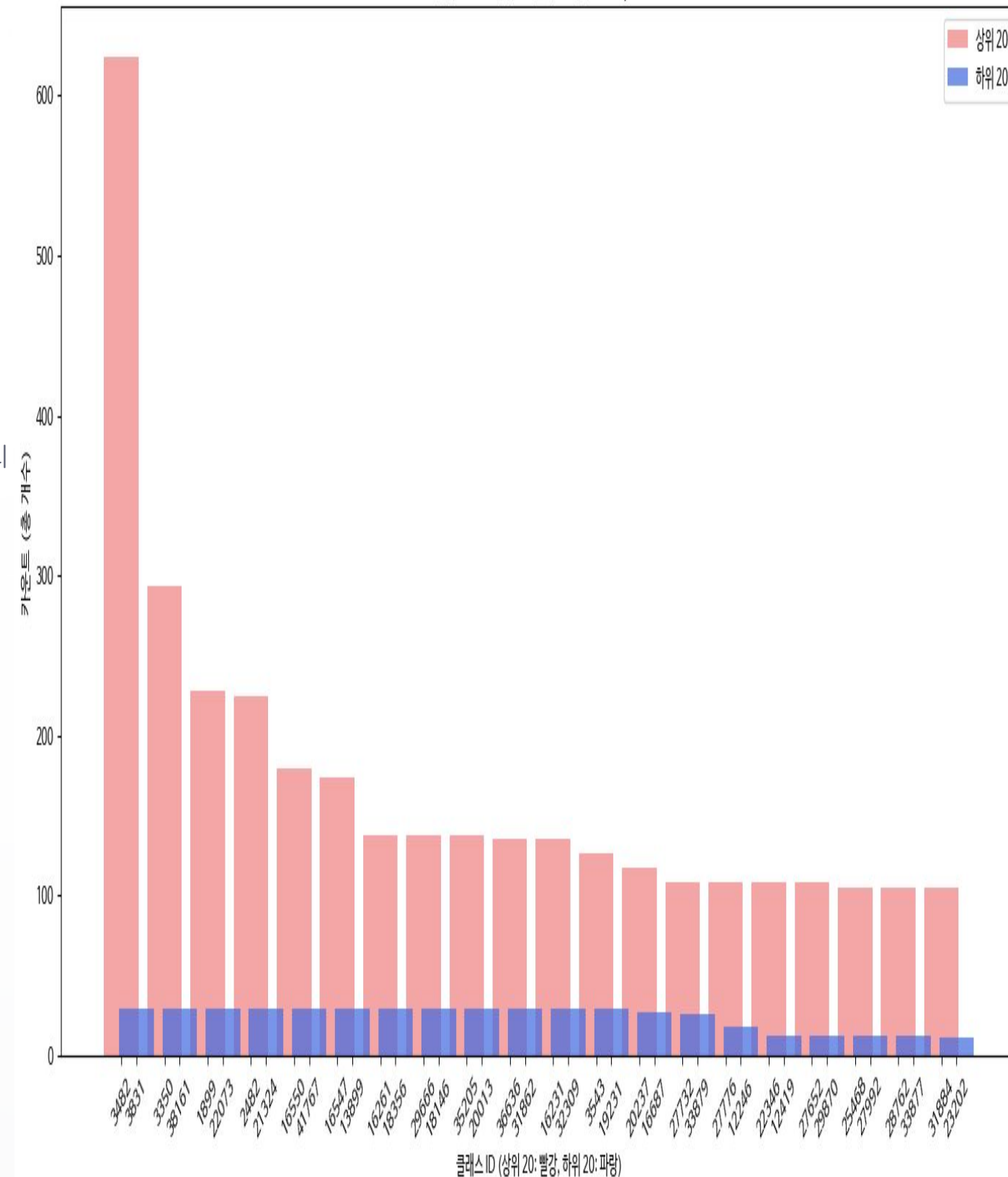
극단적인 클래스 간 편차 확인:

- **최다 클래스:** 약 600여 개의 데이터를 보유
- **최소 클래스:** 7개의 데이터만을 보유

시사점:

- 이는 최다-최소 클래스 간 **약 85배**에 달하는 극심한 데이터 불균형을 의미함.
- 모델이 소수 클래스를 학습하는 데 극도의 어려움이 예상됨.

상위 20 vs. 하위 20 클래스 카운트 Barplot



EDA 실험결과 및 분석

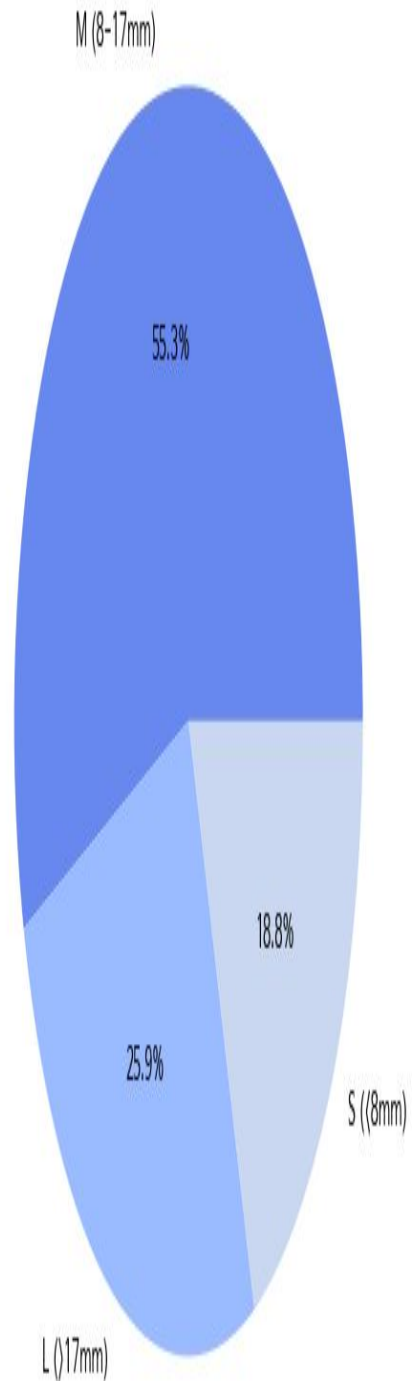
EDA (크기, 색상, 모양 비율 - 크기 불균형)

데이터의 크기별 비중, 색상별 비중, 모양별 비중을 시각화한 파이차트입니다

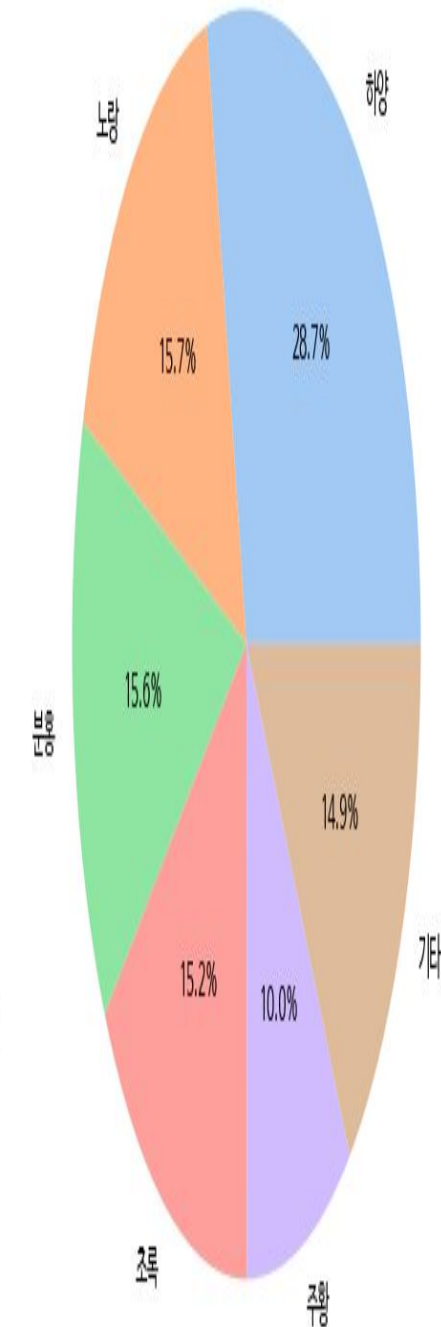
EDA (크기, 색상, 모양) 비중 분석

- 차트 분석:
 - **색상 (Color):** 대체로 균형 잡힌 분포를 보임.
 - **모양 (Shape):** '육각형' 및 '기타' 형태의 비중이 상대적으로 낮음.
 - **크기 (Size):** 'Medium'(M)이 55%로 과반수를 차지하며, 'Large'(L, 25%), 'Small'(S, 18%) 순으로 불균형함.
- 시사점 (Action Items):
 - 모델이 다양한 데이터에 대응할 수 있도록 데이터 증강 (Augmentation)이 필요함.
 - 주요 증강 대상:
 - 크기: 'Small' 및 'Large'
 - 모양: '육각형' 및 '기타' 형태

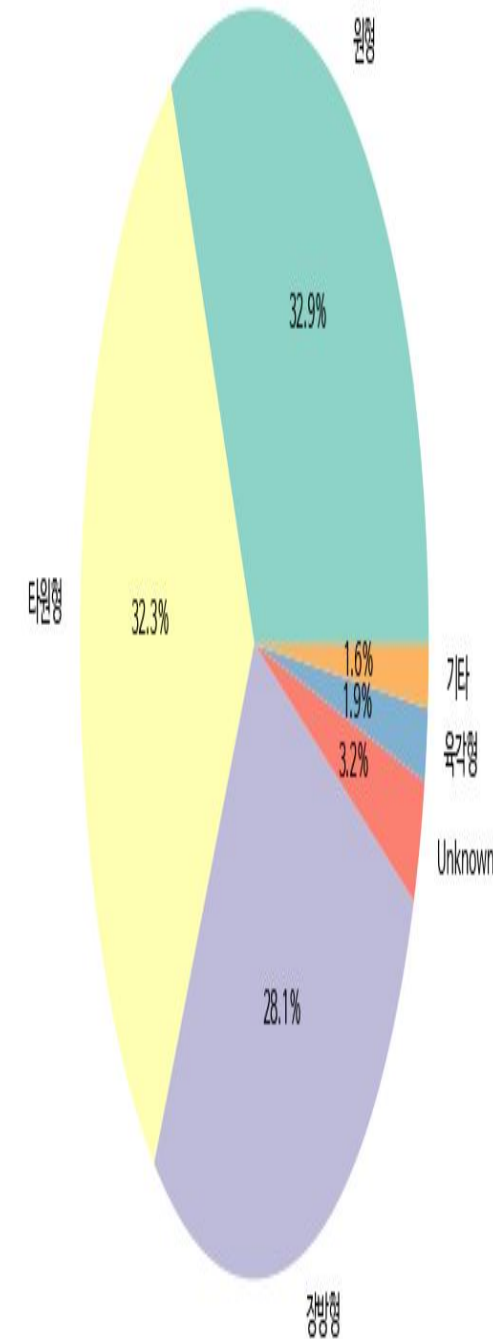
크기별 비중 (len_long mm: S/M/L)



색상별 비중 (Top 5 + 기타)



모양별 비중 (Top 5 + 기타)



EDA 실험결과 및 분석

EDA (TOP10/20 크기, 색상, 모양 비율 - 크기 불균형)

상위 20개의 클래스와 하위 20개의 클래스의 비중도 알수 있도록 시각화 하였습니다

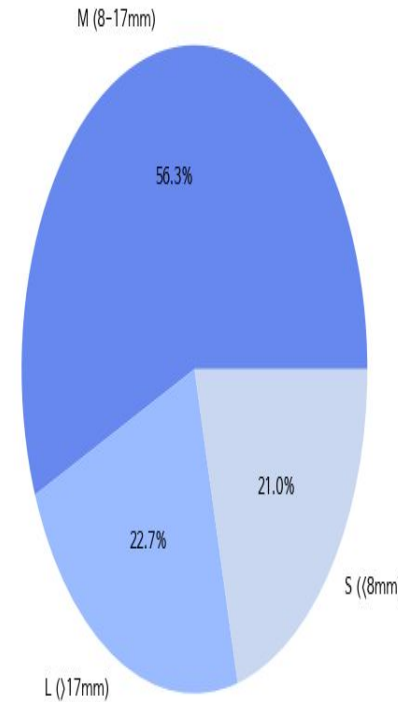
분석 대상:

- 전체 데이터의 40%를 차지하는 **상위 10개 클래스** (상단 차트)
- 전체 데이터의 60%를 차지하는 **상위 20개 클래스** (하단 차트)

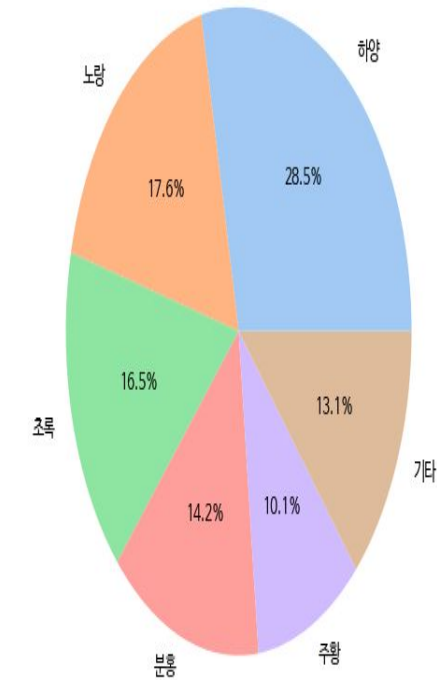
핵심 발견:

- 상위 10개 및 20개 클래스의 크기/색상/모양 비중이, 앞서 분석한 전체 데이터의 속성 비중과 대체로 유사한 패턴을 보였습니다.

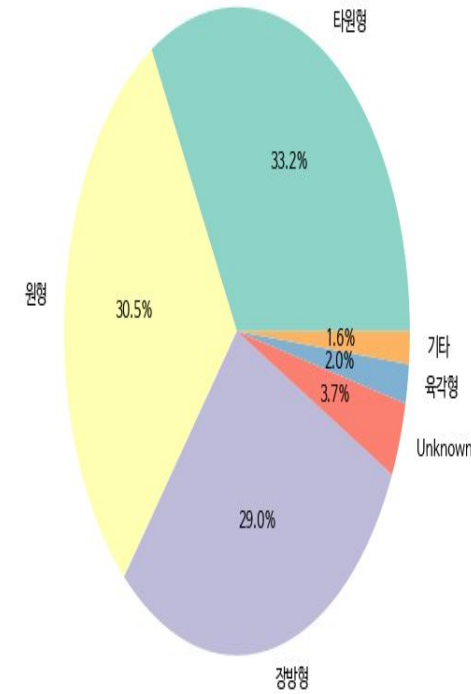
Top 10 클래스: 크기 비중



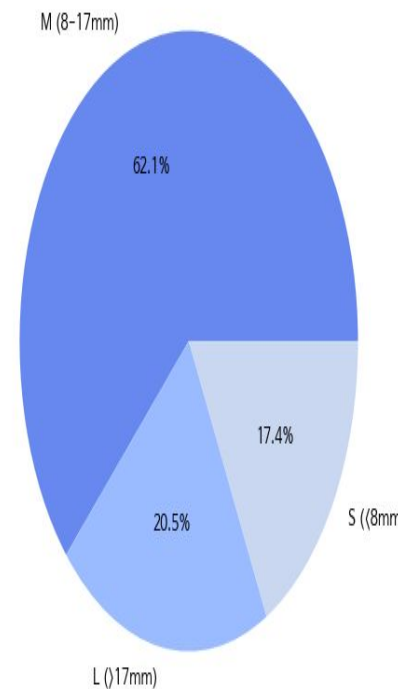
Top 10 클래스: 색상 비중



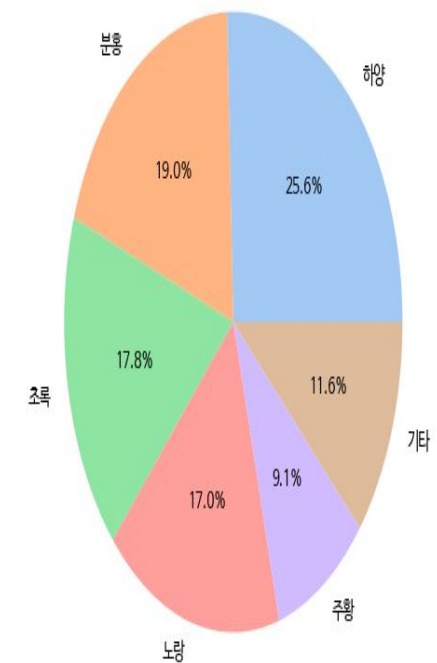
Top 10 클래스: 모양 비중



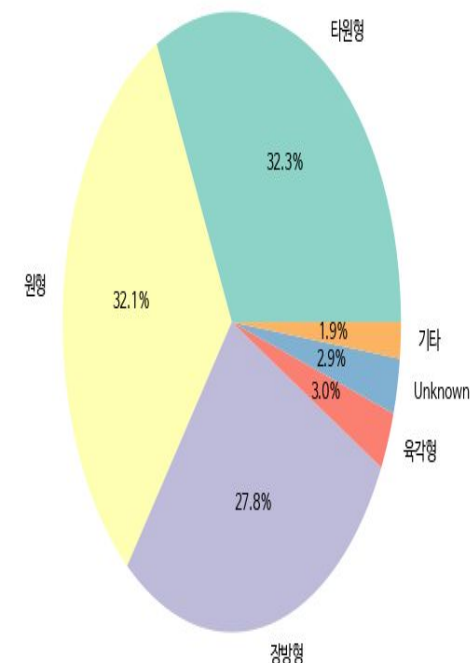
Top 20 클래스: 크기 비중



Top 20 클래스: 색상 비중



Top 20 클래스: 모양 비중



EDA 실험결과 및 분석

EDA (가로길이와 두께 분포 - 중간 크기의 알약이 많다)

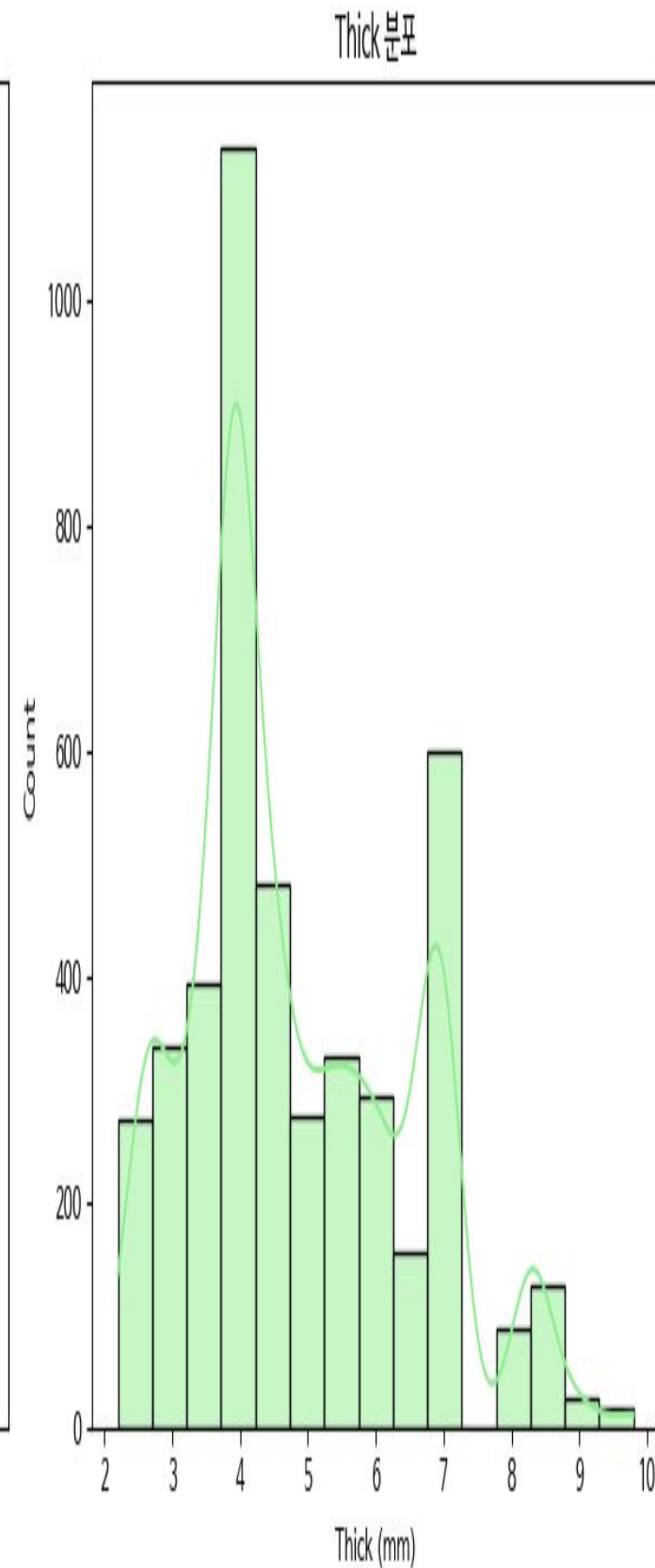
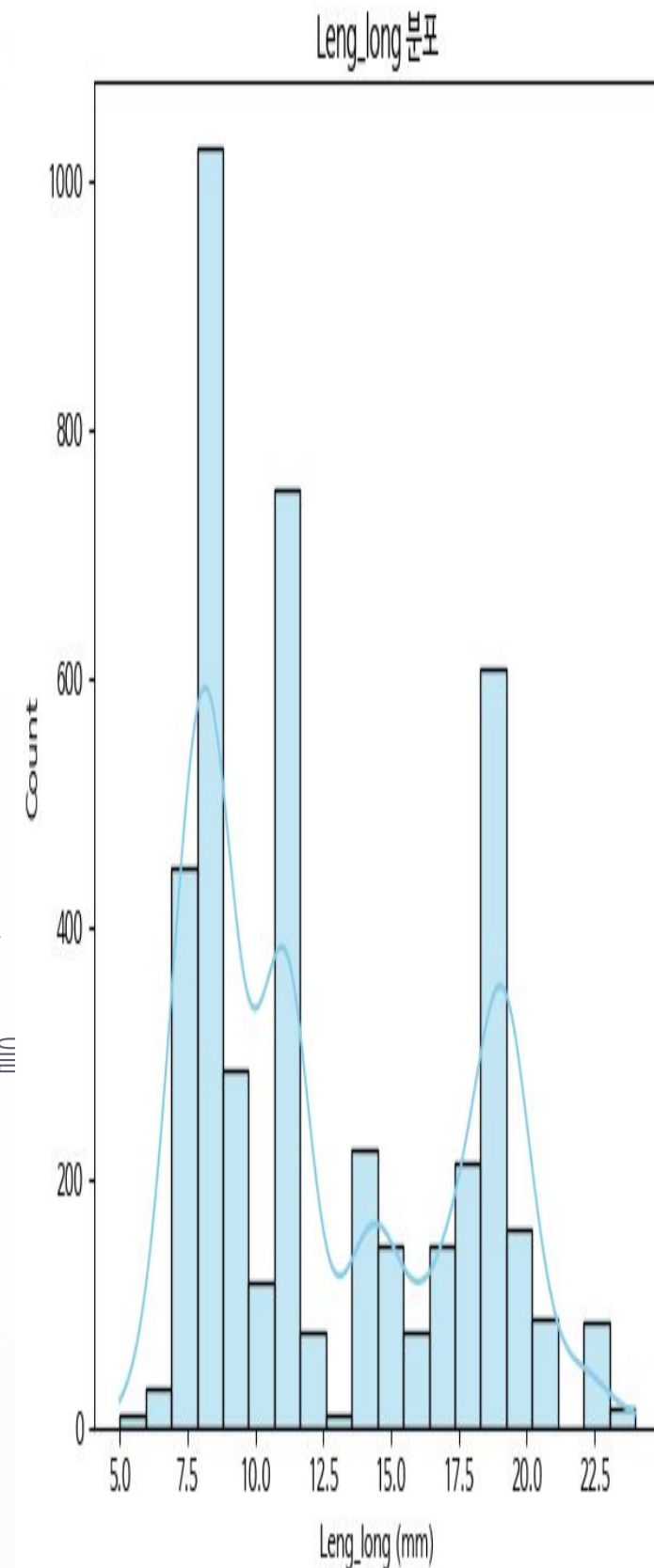
Leng_long 즉, 알약의 가로길이와 두께의 분포를 시각화 해보았습니다

차트 분석:

- 알약의 가로길이(Leng)와 두께(Thick) 분포는 모두 여러 개의 봉우리를 가진 형태를 보입니다.
- 이는 알약의 규격이 단일하지 않고, 몇 개의 **표준 그룹**으로 나뉘어 있음을 의미합니다.

핵심 발견:

- 차트의 봉우리가 특정 범위(예: 길이 7.5~18.5mm)에 집중되는 경향을 보이며, 이는 전반적으로 '중간 크기'의 알약에 데이터가 밀집되어 있음을 시사합니다



EDA 실험결과 및 분석

EDA (k-means 클러스터링)

K-means 클러스터링도 해보았습니다

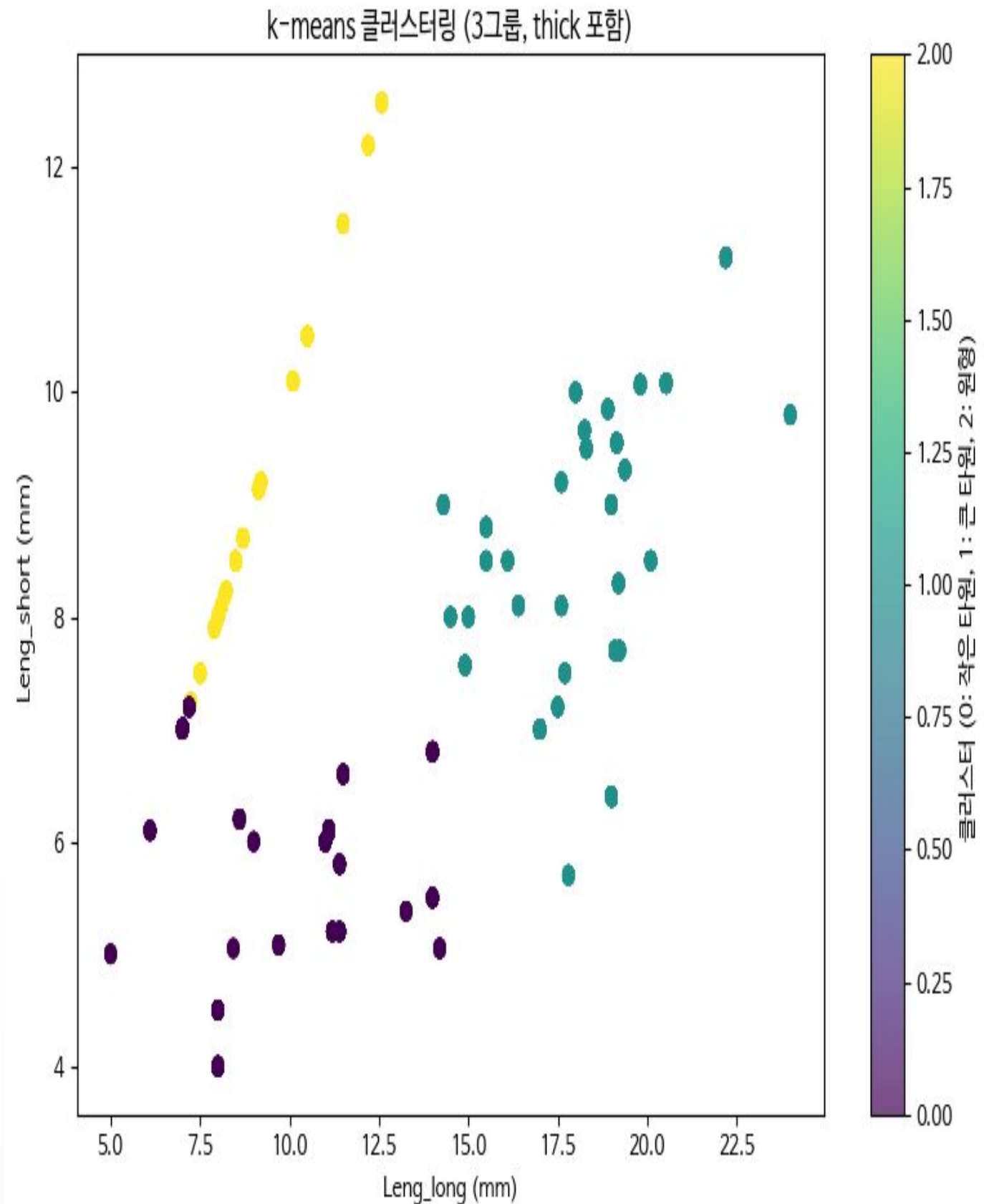
초기 관찰:

- 군집(색상)이 Y축(두께)에 따라 어느 정도 나뉘는 경향을 보여, '두께'가 유의미한 분류 기준이 될 가능성을 보임.
- (참고) 차트의 대각선($y=x$) 분포는 길이와 두께가 같은 '원형 알약' 그룹임.

핵심 발견 (한계점):

- '중간/소형(M/S) 원형 알약'(하단 대각선)들이 '중간/소형 타원형 알약' 군집과 분리되지 않고 동일한 군집(보라색/녹색)으로 묶이는 문제가 발생함.

결론 및 시사점:



데이터 전처리 BBox 오류

제거

EDA (GT BBOX이상, 오류, 중첩)

데이터 중 과도한 이상치들은 제거를 진행했습니다.

① 과도하게 넓거나 어긋난 GT BBOX 제외

② 중첩 영역은 정답 클래스만 남기고 제외



데이터 로딩 오류 발견

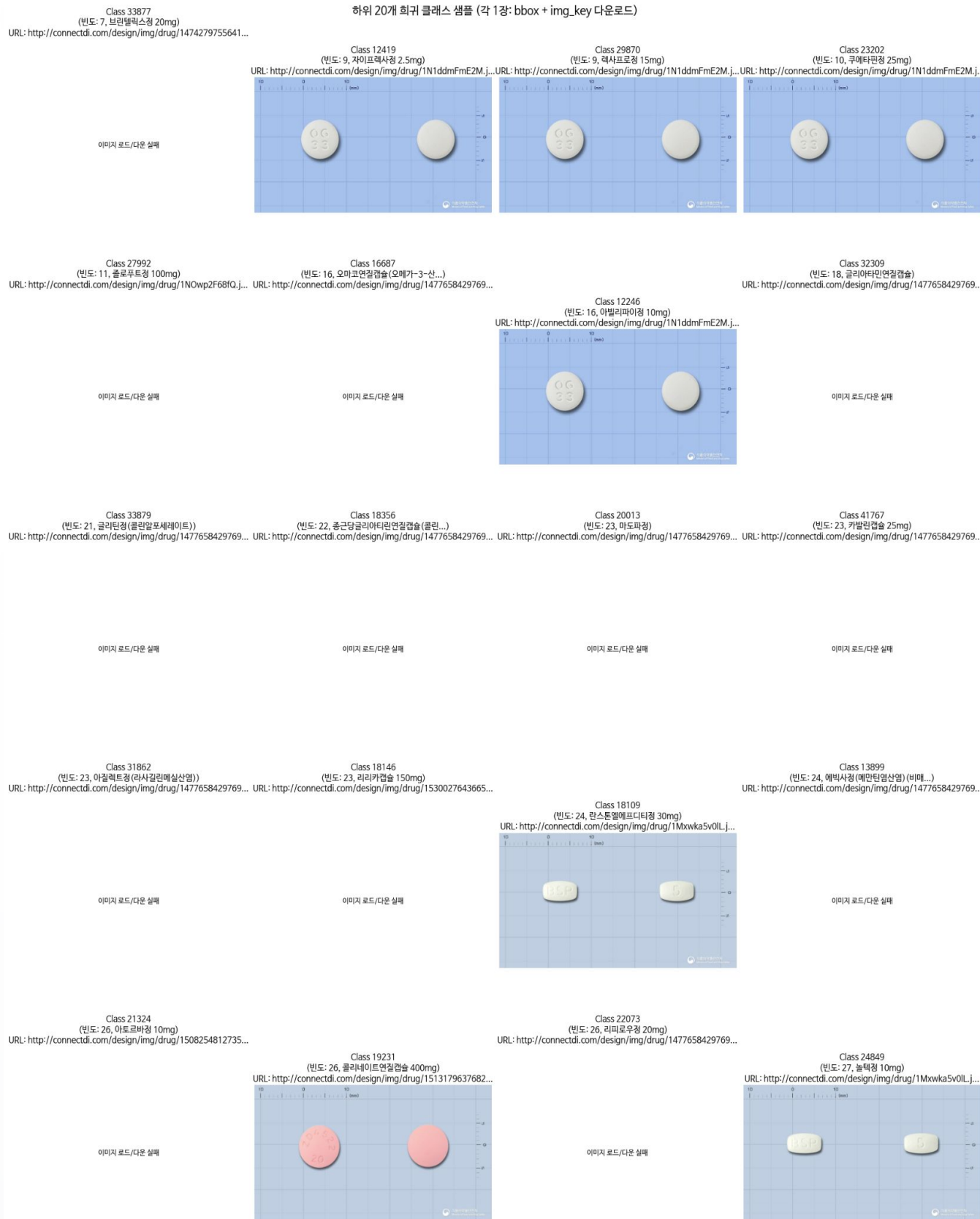
EDA (어노테이션 img_key의 404오류)

라벨의 img_key 영역을 통해 이미지를 로딩해서 클래스 분류를 빠르게 구분하려고 했었는데 404 에러가 떠서 가져올 수 없었습니다.

문제 발견:img_key를 통해 자동 로딩 실패(404) → 데이터 클래스 분류가 정확하게 이루어졌는지 1000장이 넘는 이미지를 확인하고 라벨링 하는데 어려움을 겪음

해결 방안: 커스텀 모델 기반 오토라벨링 도입

목적: 라벨링 효율성 향상 및 품질 개선



오토라벨링 (CVAT)

적용

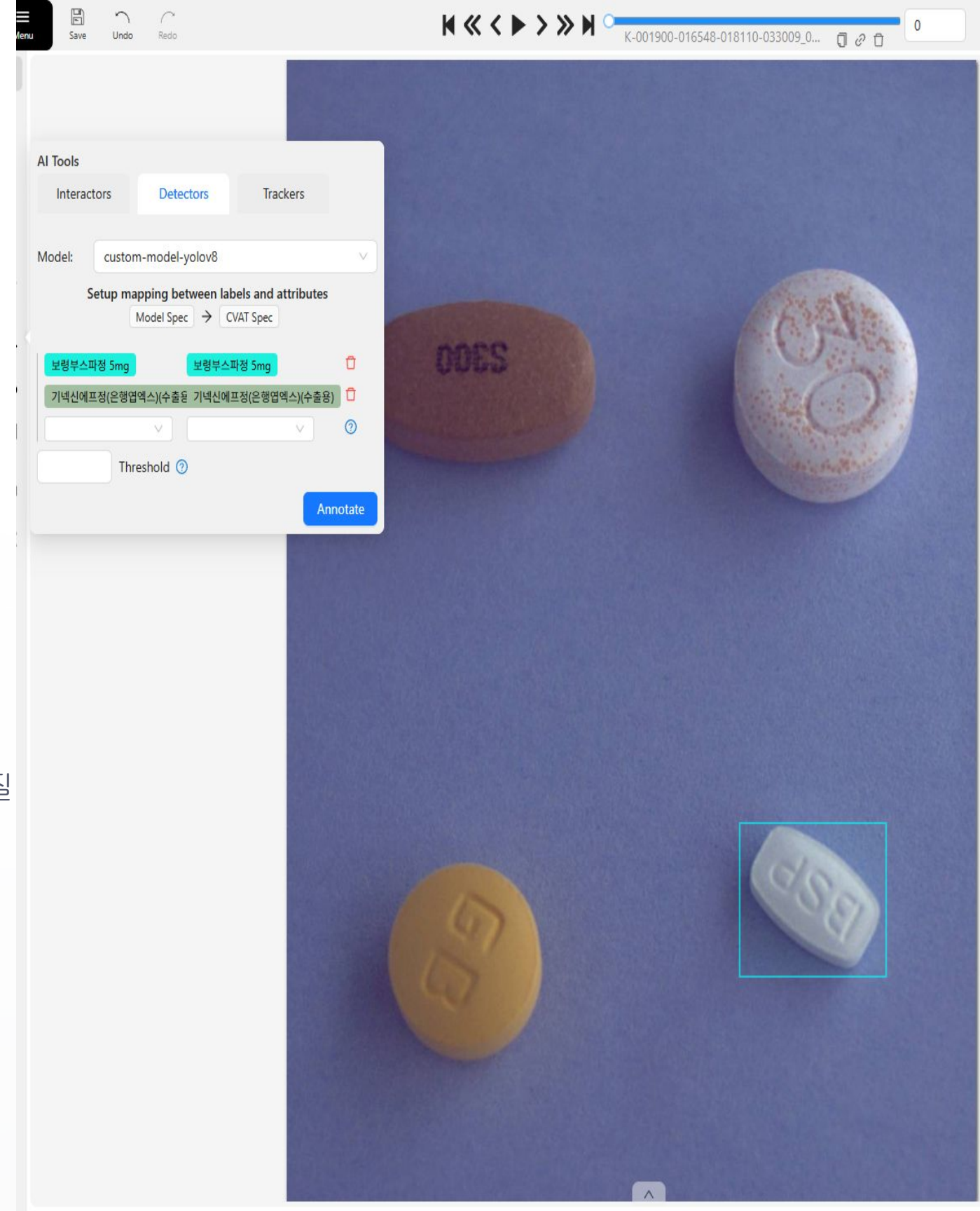
오토라벨링 (CVAT)

오토 라벨링은 오픈소스인 CVAT를 사용했습니다

CVAT 오토라벨링 적용 (Custom YOLOv8 활용)

라벨 누락 데이터에 모델 기반 자동 라벨링 적용

자동 생성된 라벨을 사람이 검토 및 수정하여 라벨링 효율성과 최종 품질 향상



BBox 파이프라인 구축

표준화된 실험 환경 (src/)

모든 BBox 실험을 공정하고 재현 가능하게 수행하기 위해, src/ 기반의 표준화된 파이프라인을 먼저 구축했습니다.

파이프라인 핵심 기능

일관된 훈련/평가: 모든 실험이

동일한 yolo train.py 기반

스크립트를 사용

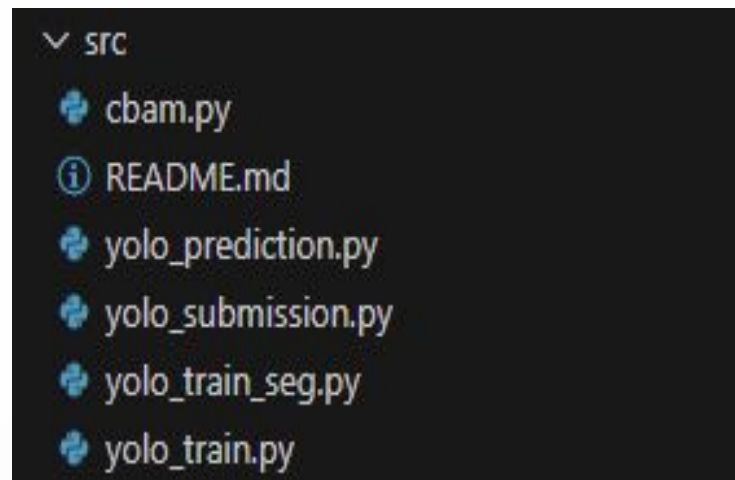
재현 가능한 실험 관리:

yolo train config.yaml 파일로 모든

하이퍼파라미터를 명시적으로 관리

자동화된 결과 로깅: mAP, Loss 로그

및 모델 가중치를 체계적으로 저장



```
# yolo_train_config.yaml

# Train
model: yolo11n.pt
data: data.yaml
epochs: 300
patience: 50
batch: 0.9
imgsz: 1280

device: 0

project: 'runs'

# Hyperparameters
#lr0: 0.001

# box: 6.0 # Box Loss Weight: 날개로 떨어져 있어 Box 정확도 중요도를 상대적으로 낮춤 (기본 7.5)
# cls: 1.5 # Cls Loss Weight: 73개 클래스 구분을 위해 분류 정확도에 집중 (기본 0.5)
# dfl: 1.5 # DFL Loss Weight: 기본값 유지

hsv_h: 0
hsv_s: 0
#hsv_v: 0
#degrees: 180

mosaic: 1.0

# lrf: 0.01 # 최종 학습률 (최소한의 가중치 업데이트만 남김)
# momentum: 0.937 # SGD 모멘텀 (변경 없음)
# weight_decay: 0.0 # L2 정규화 (가중치 감소)를 완전히 제거하여 과적합 허용
# dropout: 0.0

# hsv_h: 0.0 # HSV Hue (색조) 증감 비율
# hsv_s: 0.0 # HSV Saturation (채도) 증감 비율
# hsv_v: 0.0 # HSV Value (명도) 증감 비율
# degrees: 0.0 # 이미지 회전 각도
# translate: 0.0 # 이미지 이동 비율
# scale: 0.0 # 이미지 스케일 비율
# shear: 0.0 # 이미지 전단(Shear) 비율
# perspective: 0.0 # 이미지 원근 변환
# flipud: 0.0 # 상하 반전 확률
# fliplr: 0.0 # 좌우 반전 확률
# mosaic: 0.0 # 모자이크 증감 확률
# mixup: 0.0 # MixUp 증감 확률
# copy_paste: 0.0 # Copy-Paste 증감 확률
```

Baseline 수립 및 아키텍처 결정

Baseline 모델 수립

Baseline으로 YOLOv8n 모델을 사용하여 초기 mAP 0.98199 확보

초기 시도

- n(nano) 모델을 시도하다 73개의 클래스에 대응하려면 더 큰 모델이 필요할 것 같아 s(small) 모델로 상향 조정

실험 결과

- 오히려 YOLOv8s에서 YOLOv8n 보다 성능이 소폭 감소할 수 있고 학습 속도는 훨씬 많이 소요됨을 확인. 추가적인 복잡성이 큰 이점을 주지 못했음.

결론

- 간단한 알약 이미지는 가벼운 모델로 충분히 학습 가능하고, 모바일앱이라는 프로젝트 특성상 **가벼운 모델 사용 결정**

아키텍처 비교 테스트

알약의 클래스 수가 매우 많지만,
해당 데이터셋은 경량 모델
(YOLOv8n)에서 더 효율적인 성능을
보였음.

모델 (아키텍처)	mAP	캐글 점수	학습 속도 (시간)
YOLOv8n	0.8622	0.98199	415s
YOLOv8s	0.86863	0.97735	7374s
YOLOv11n	0.88425	0.98735	2907s

YOLOv8n의 실시간 탐지 PoC

Android 기기 자체에서 YOLOv8n 모델을 이용한 객체 탐지 실시간 추론
가능성 입증.

모바일 환경에서 상용화 가능한 수준의 속도

- YOLOv8n은 모바일 환경에서도 속도와 정확도를 만족
- 모바일 환경에 최적화된 엔진인 TensorFlow Lite로 변환
- 후처리 통해 화면에 바로 알약의 이름과 Box를 표기 가능

My Application



탐지 완료! 총 4개 객체 탐지.

Augmentation

선택적 데이터 증강 적용

알약의 고유한 식별 특징을 유지하고, 데이터셋의 특징에 맞는 증강을 선택적으로 적용

1. 색 공간 증강

- `hsv_h`(색조, Hue): 알약의 종류를 구분하는 가장 핵심적인 특징이 색상이기에 0으로 비활성화
- `hsv_s`(채도, Saturation): 알약의 표면 코팅이나 재질 특성을 유지하기 위해 0으로 비활성화
- `hsv_v`(밝기, Value/Brightness): 조명 변화에 대해 강건성 확보가 될 수 있지만 일관된 데이터셋 특성상 낮게 유지

2. 기하학적 변환

- `flipud`, `fliplr`(반전): 알약이 뒤집히면 글자등의 정보가 손실되므로 일부러 적용하지 않음.
- `degrees`(회전): 알약이 놓인 방향이 다른 경우를 상정해 180° 를 적용해보았으나 `bbox` 값이 오염되는지 오히려 성능이 떨어져 0으로 비활성화함.
- `mosaic`(모자이크): 4개의 훈련 이미지를 하나로 결합하는 변환. 비활성화하니 다양한 훈련 샘플을 생성하지 못하고 제한된 데이터만 사용해 학습하여 성능이 떨어짐

BBox 모델 최적화

모델 튜닝 및 증강 실험

결정된 Baseline 모델의 성능을 더욱 향상시키기 위해,
하이퍼파라미터 튜닝과 데이터 증강 실험을 진행했습니다

하이퍼파라미터 튜닝

imgsz(이미지 크기), epochs, lr(학습률),
batch 등 주요 하이퍼파라미터를
체계적으로 조정하여 최적의 조합을
탐색했습니다.

lr	imgsz	epoch	patience	batch
0.1	640	50	25(채택)	8
0.01	960(채택)	100(채택)	50	16(채택)
0.001(채택)	1280	150	100	-

데이터 증강 실험

BBox 모델의 성능을 높이기 위해 HSV,
Mosaic, Rotation 등 다양한 데이터 증강
기법의 적용 유무를 테스트했습니다.

실험 조건	mAP	결론
Baseline	0.97	-
Mosaic	0.96	성능 하락
Rotation	0.94	성능 하락
HSV (채택)	0.98	성능 향상

BBOX의 문제점 발견 (GT 비일관성)

BBOX 문제점 발견

캐글을 이용해 평가를 진행하였으나 문제가 발생하였습니다

문제 상황

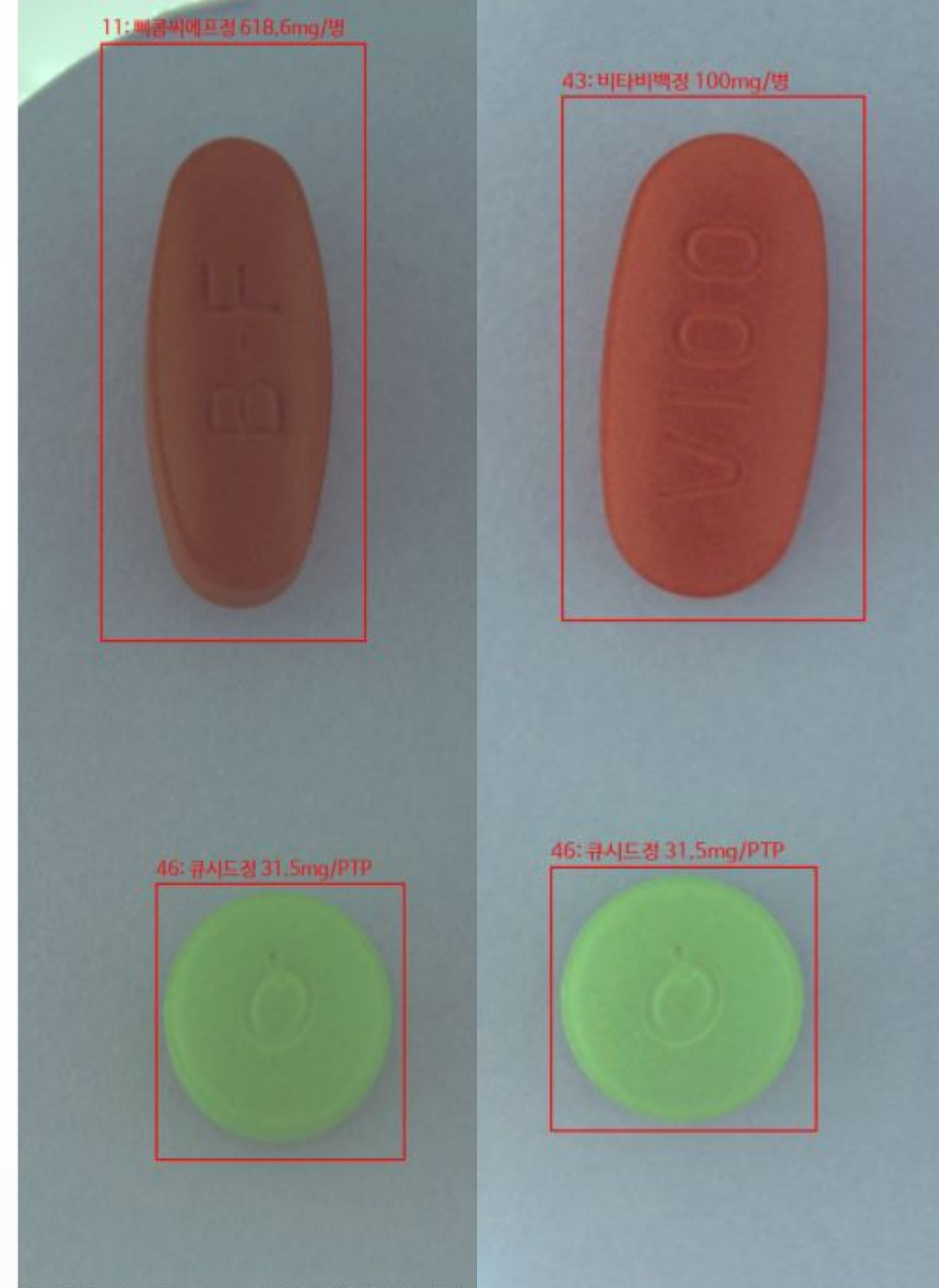
정제를 위해 '오토 라벨링'을 적용했으나, 오히려 Kaggle 점수가 **하락**하는 문제 발생.

원인 분석

1. **BBox의 형태적 한계**: 원형/타원형 알약은 사각형 BBox가 **불필요한 배경까지 정답으로 포함**시킴.
2. **GT(정답)의 비일관성**: 원본 GT 라벨 자체가 타이트/혈령/오류 등 **일관성이 매우 부족**했음.
3. **"개선의 역설"**: 엄격한 mAP 지표가 '깨끗하게 수정된 라벨'을 '오류가 있는 원본 GT'와 다르다는 이유로 오히려 옳다고

최종 결론

- BBox 방식은 알약 식별에 근본적인 한계가 있음을 확인함.
- Kaggle 점수를 위해, 아이러니하게도 정제된 데이터가 아닌 '오류가 있는 원본 데이터'에 **과적합 (Overfitting)**시키는 전략을 채택할 수밖에 없었음.



둘다 원본 GT BBOX

결정적 한계 발견 (데이터

누수)

데이터 누수 문제 발견

Train_images내에서 Test_images의 대부분이 발견됨

테스트 셋과 유사한 이미지를 대상으로 Loss 값을 높임으로써 더 학습이 잘되게끔 유도하는 데이터 전처리 실험과정에서, train_images 폴더 내에서 test_images와 동일한 이미지가 존재함을 확인했습니다.

이로 인해 모델의 실제 성능보다 더 높은 결과(과도한 성능)가 측정되었습니다

전략 전면 수정 결정

BBox 한계를 인지하고,
Segmentation 기반 접근으로
전환합니다.



테스트셋 이미지



트레인셋 이미지(유사)



트레인셋 이미지(동일)

SAM 을 이용한 seg 데이터 추가

SAM을 활용하여 부족한 데이터셋 보강

데이터셋 내의 각 알약들의 마스크를 따서 개별 저장
마스킹 된 알약들을 기반으로 알약이 제거된 빈 배경 생성
생성 된 빈 배경에 마스크 된 알약을 3~4개 씩 무작위 합성



Segmentation 데이터셋 구축

SAM 활용 고품질

데이터 생성

BBox의 한계를 극복하기 위해

SAM을 활용, 픽셀 단위의

정밀한 Segmentation

데이터셋을 직접 구축했습니다.



CBAM

YOLO 아키텍처에 CBAM 모듈을 커스텀하여 통합했습니다

목적 : 약제의 중요한 특징(글자, 모양)에 **선택적으로 집중** 하고 불필요한 배경 정보는 억제

작동 원리 : 원본 특징을 입력받아 무엇이 중요한지와 어디가 중요한지를 순차적으로 학습하여 **핵심 영역에 가중치를 부여**

최종 효과 : 중요한 특징만 강화된 피처맵을 생성하여, 최종 모델의 탐지 정밀도를 극대화

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ChannelAttention(nn.Module):
    def __init__(self, in_planes, ratio=16):
        super(ChannelAttention, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)
        self.fc1 = nn.Linear(in_planes, in_planes // ratio, bias=False)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(in_planes // ratio, in_planes, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = self.fc2(self.relu1(self.fc1(self.avg_pool(x).squeeze(-1).squeeze(-1))))
        max_out = self.fc2(self.relu1(self.fc1(self.max_pool(x).squeeze(-1).squeeze(-1))))
        out = avg_out + max_out
        return self.sigmoid(out).unsqueeze(-1).unsqueeze(-1)

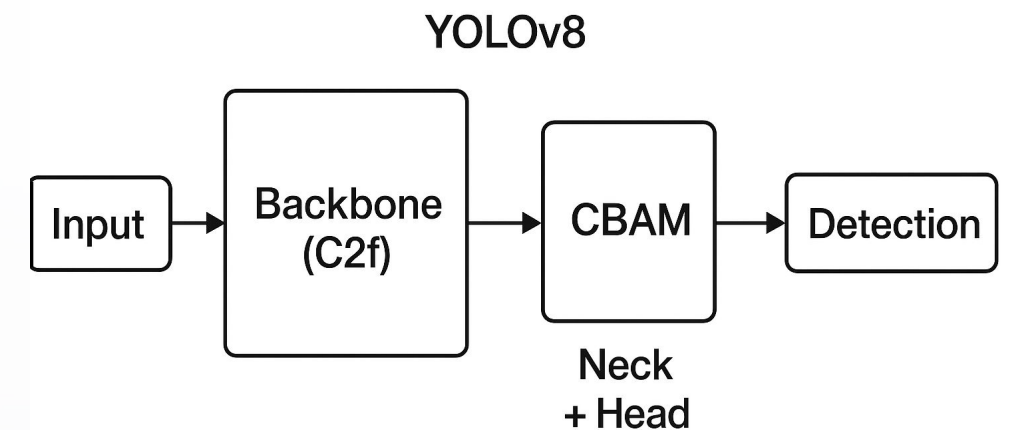
class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super(SpatialAttention, self).__init__()
        self.conv1 = nn.Conv2d(2, 1, kernel_size, padding=kernel_size//2, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        x = torch.cat([avg_out, max_out], dim=1)
        x = self.conv1(x)
        return self.sigmoid(x)

class CBAM(nn.Module):
    def __init__(self, planes, ratio=16, kernel_size=7):
        super(CBAM, self).__init__()
        self.ca = ChannelAttention(planes, ratio)
        self.sa = SpatialAttention(kernel_size)

    def forward(self, x):
        x = x * self.ca(x)
        x = x * self.sa(x)
        return x
```

CBAM 모듈 구현 코드 (cbam.py)



YOLO 아키텍처 내 CBAM 적용 위치 (다이어그램)

데이터 증강 최적화

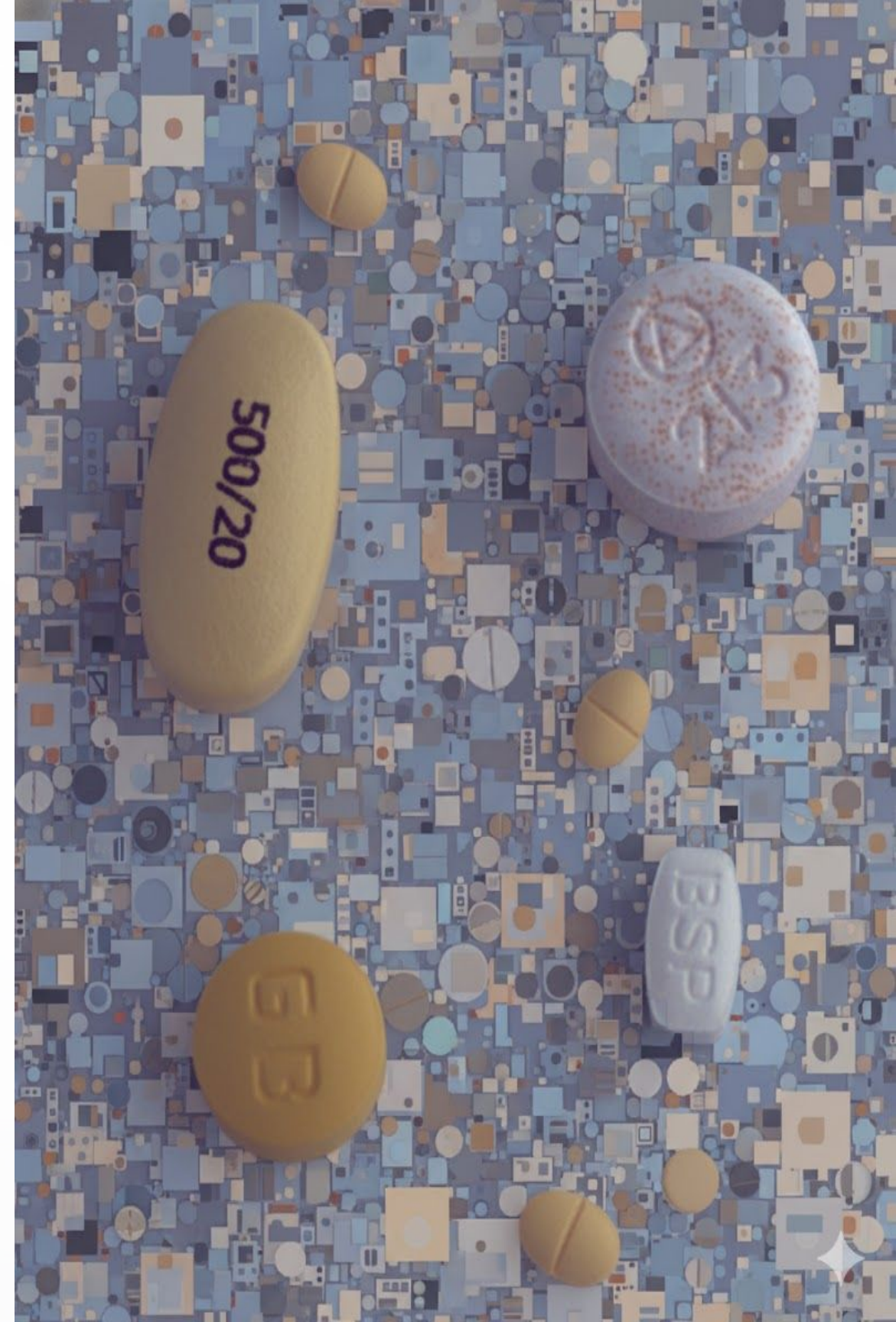
SAM+Diffusion 기반 배경 합성 증강

모델의 강건성을 확보하기 위해 Segmentation에 특화된 데이터 증강 기법을 개발하고 적용했습니다.

배경 합성 기법

- SAM 마스크로 약제 객체 분리
- 다양한 배경 이미지에 합성
- 조명 및 각도 변화 시뮬레이션
- 자연스러운 그림자 효과 추가

실제 환경의 다양성을 반영하여
모델이 어떤 조건에서도 안정적으로
작동하도록 훈련했습니다.



시스템 구축 및 실험



테스트셋 이미지

← 탐지 결과



비모보정 500/20mg (신뢰도: 0.66
좌표: [[936.7923583984375,
928.6105346679688, 1895.112548828125,
2613.63818359375]])

오탐지 유도 이미지(배경수정, 촬영각도 조절) + 모바일 앱 탐지

실시간 서빙 시스템

개발 모델을 실제 서비스 환경에 통합했습니다.

- 고성능 API 서버 구축
- 모바일 앱 실시간 연동

모바일 앱

약제 촬영

FastAPI 서버

추론 요청

AI 모델

Segmentation 및 결과 출력

결과 표시

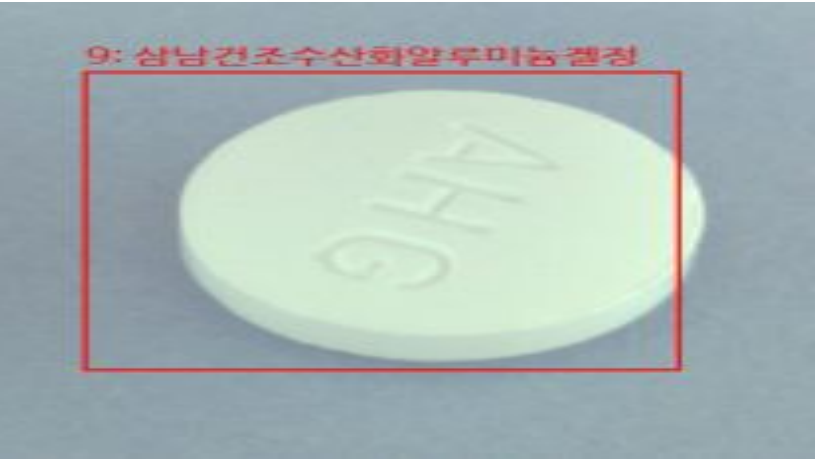
정보 제공

핵심 결과: BBox vs Segmentation 성능

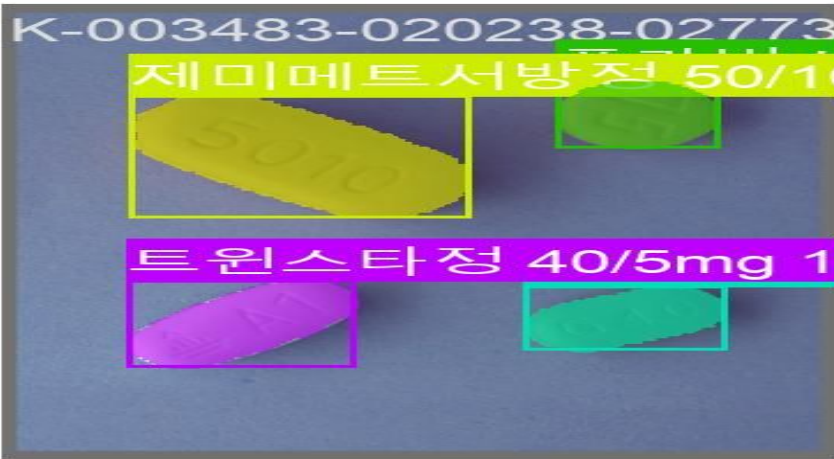
비교

Segmentation (CBAM) 모델은 BBox Baseline 대비 향상된 성능과 복잡한 약제 이미지 분석에서 더 정확한 결과를 제공합니다.

성능 지표	BBox Baseline 모델	Segmentation (CBAM) 모델
mAP50-95	0.87013	0.99022
F1-Score(Precision, Recall)	0.8520(0.7716, 0.9512)	0.9704(0.9622, 0.9787)



BBox 실패: 경계 상자 겹침 및 부정확한 인식



Segmentation 성공: 픽셀 단위 분할로 정확한 인식

최종 성과 및 결론

핵심 성과

- 데이터 문제 해결 및 새로운 데이터셋 구축
- 모델 성능 향상 기법 적용
- 실시간 서빙 시스템 및 모바일 앱 연동

핵심 교훈

- 데이터 품질의 중요성 확인
- 문제 해결을 위한 전략적 접근
- 성공적인 프로젝트를 위한 팀워크

경구약제 식별 정확도 개선 및 서비스 시스템 구축을 성공적으로 완료했습니다.

Q&A

궁금한 점이 있으시면 자유롭게 질문해주세요.